Gio Vase, Dashboard

Release 1.8

Author: Monica Amico

Jun 25, 2020

CONTENTS:

CHAPTER

ONE

MODEL, CONSTANT

class model.constant.**Operation**(*value*)

Enum used to represent Type of operations:

- JOINED = 0
- REFUSED = 1
- PING = 2
- HUMIDITY = 3
- TEMPERATURE = 4
- LIGHT = 5
- WATER = 6
- SET_HUMIDITY_MIN = 7
- SET_HUMIDITY_MAX = 8
- SET_TEMPERATURE_MIN = 9
- SET_TEMPERATURE_MAX = 10
- SET_LIGHT_MAX = 11
- SET_LIGHT_MIN = 12
- SET_VASE_PAUSE_TIME = 13
- SET_VASE_SEND_TIME = 14
- SET_RADIO_PAUSE_TIME = 15
- SET_RADIO_DIED_PING = 16
- CONNECTION = 17
- DELETED = 18
- SET_WATERING_LIGHT = 19
- SET_WATER_CONTAINER_SIZE = 20
- WATER_CONTAINER_STATE = 21
- RADIO_JOIN = 22
- RADIO_TRANSMIT_POWER = 23
- VASE_TRANSMIT_POWER = 24

class model.constant.**Role**(*value*)

Enum used for users role:

- ADMIN = 0
- USER = 1

class model.constant.**WaterContainerState**(*value*)

Water container state:

- Empty = 0
- Full = 1

CHAPTER

TWO

MODEL, DATABASE

class model.gio_db.**ConnectionRequest**(***kwargs*)
A class used to represent a connection request from plant

Params:

- id : String, vase serial number
- signal: int, the value of the request's signal, ranges from -128 to -42. -128 means a weak signal and -42 means a strong one.
- pairing: int, three-digit random integer, representing the connection request of the plant.
- radio_id : String, radio serial number

class model.gio_db.**Plant**(***kwargs*)
A class used to represent the plant

class model.gio_db.**Radio**(***kwargs*)
A class used to represent the radio-raspberry

Params:

- id: String, radio serial number
- name: string, radio name
- url_radio: string, raspberry url used to communicate operation request to the radio and to the associated plants
- transmit_power : int, radio transmission power, range 0-7 (lower, higher)
- sleep_time: time interval in which the radio is in sleep state (minutes)

class model.gio_db.**TypePlant**(***kwargs*)
Represent the type of plant

class model.gio_db.**User**(**args*, ***kw*)
Represent one user.

Params:

- id
- username
- password
- role
- is_active: boolean
- is_anonymous: boolean

authenticate(*password*)
if the password provided is correct, it authenticates the user :param: password: user password :type: password: string :return: True or False

get_id()

Returns the user id

Return type Integer

property is_admin
check if the user is admin :return: True or False

is_authenticated()
it returns true if the user is already authenticated, otherwise false :return: self.is_authenticated :rtype: true or false

set_password(*password*)
set the password of the user :param: password: user password :type: password: string

set_role(*role*)
It sets the user role :param role: the role :type role: value of the enum Role :return: True if the user's role has been set, otherwise false :rtype: Boolean

set_username(*username*)
it sets the user username :param username: new username :type username: string :return: true if username has been set, otherwise false :rtype: Boolean

CHAPTER

THREE

CONTROLLER, UTILITY FUNCTIONS

controller.utility.**add_conn_req**(*idv*, *signalv*, *pairingv*, *radio_id*)

- **if there is no connection request or a plant with id equal to the idv parameter:** it adds the connection request and returns the value True.
- **If the connection request with idv was already present:** controls the signal strength of both requests (new and old). If the signal strength of the new request is greater, it replaces the old one with the new one and sends the refusal operation to the radio of the old connection request, otherwise only the pairing number changes. In both cases the function returns the value True.
- **If there was already a plant with id equal to idv (parameter):** send the join request to the associated radio: if this request is not successful, the plant is deleted and the new connection request is inserted, otherwise it does nothing. In this case the function returns the value False.

Parameters

- **idv** (*string*) – vase serial number
- **signalv** (*int*) – the value of the request's signal.
- **pairingv** (*int*) – three-digit random integer
- **radio_id** (*string*) – radio serial number

Returns True - if there was no connection request with id equal to idv, or if it was present, but it has been replaced because the signal of the new request is stronger. False - if a plant with id equal to idv was present and cannot communicate with the associated radio.

Return type boolean

controller.utility.**add_plant**(*idv*, *radio*)

if the radio with serial number equal to the second parameter exists and the plant doesn't exist, adds the plant with serial number equal to idv, and deletes the connection request with id equal to idv.

Parameters

- **idv** (*string*) – plant serial number
- **radio** (*string*) – radio serial number

controller.utility.**add_radio**(*radio*, *url_radio*)

- **If the radio is not in the database:** add it, and return the value True.
- **If the radio is already present:**
  - if the url has not changed: it does nothing and returns the False value,
  - if the url has changed: change url and reset all the parameters, return the False value

Parameters

- **radio** (*str*) – serial number
- **url_radio** (*str*) – raspberry (radio) url

Returns True or False

Return type bool

controller.utility.**add_type**(*idt*, *hum_min*, *hum_max*, *temp_min*, *temp_max*, *light_max*, *light_min*)

Adds a new type of plant

Parameters

- **idt** (*str*) – id (name) of the type
- **hum_min** (*int*) – min. humidity of the type
- **hum_max** (*int*) – max. humidity of the type
- **temp_min** (*int*) – min. temperature of the type
- **temp_max** (*int*) – max. temperature of the type
- **light_max** (*int*) – max. light of the type
- **light_min** (*int*) – min. light of the type

controller.utility.**add_user**(*username*, *password*, *role*)

Add a user into the database

Parameters

- **username** –
- **password** –
- **role** –

:return True if the operation was successful otherwise False

controller.utility.**associated_plants**(*radio_id*)

Gets plants list associated with the radio

Parameters **radio_id** – radio serial number

Returns the list of plants associated with the radio

Return type plant list

controller.utility.**change_type**(*idv*, *idt*, *url*)

Change the type of the plant

Parameters

- **idv** (*str*) – plant serial number
- **idt** (*int*) – type of plant id
- **url** (*str*) – associated radio's url

Returns True in case of success, otherwise False

Return type bool

controller.utility.**correct_password_user**(*username*, *password*)

Parameters

- **username** (*string*) –
- **password** (*string*) –

Returns True if the user's password is correct otherwise False

Return type boolean

controller.utility.**delete_conn_req**(*idv*, *radio*)
if present, it eliminates the connection request with id equal to the first parameter from the radio with id equal to the second parameter.

Parameters

- **idv** (*string*) – connection (plant) serial number
- **radio** (*string*) – radio serial number

Returns None or the deleted request

controller.utility.**delete_plant**(*idv*)
Delete the plant with serial number equal to idv, if presents.

Parameters **idv** (*str*) – plant serial number

controller.utility.**delete_radio**(*radio*)
Delete the radio with serial number equal to parameter

Parameters **radio** (*str*) – serial number

Returns True or False

Return type bool

controller.utility.**delete_user**(*username*, *password*)
Delete the user username.

Parameters

- **username** (*str*) –
- **password** (*str*) –

Returns True if the operation was successful otherwise False

Return type bool

controller.utility.**get_user**(*username*)

Parameters **username** – user's username

:return the user with username equal to the parameter, if it exist, otherwise none.

controller.utility.**radio_from_url**(*url*)
Gets the radio with url equal to parameter.

Parameters **url** (*str*) – raspberry-radio url

Returns object radio or None

Return type radio

controller.utility.**update_hum**(*idv*, *humidity*)

Parameters

- **idv** (*str*) – plant serial number
- **humidity** (*int*) – humidity measure

controller.utility.**update_hum_max**(*idv*, *hum_m*)

controller.utility.**update_hum_min**(*idv*, *hum_m*)

controller.utility.**update_ideal_hum**(*idv*, *ideal_humidity*)

Parameters

- **idv** (*str*) – plant serial number
- **ideal_humidity** (*int*) –

controller.utility.**update_ideal_light**(*idv*, *ideal_light*)

Parameters

- **idv** (*str*) – plant serial number
- **ideal_light** –

controller.utility.**update_ideal_temp**(*idv*, *ideal_temp*)

Parameters

- **idv** (*str*) – plant serial number
- **ideal_temp** –

controller.utility.**update_light**(*idv*, *light*)

Parameters

- **idv** (*str*) – plant serial number
- **light** (*int*) – light measure

controller.utility.**update_light_max**(*idv*, *li_m*)

controller.utility.**update_light_min**(*idv*, *li_m*)

controller.utility.**update_name**(*idv*, *name*)

Change the plant's name.

Parameters

- **idv** (*str*) – plant serial number
- **name** (*str*) – new name

Returns True (success) or False

Return type bool

controller.utility.**update_plant_state_fitness**(*idv*)

Update the plant status, calculated with the values of:

- ideal humidity, ideal light, ideal temperature
- current humidity, current light, current temperature

Parameters **idv** (*str*) – serial number

Returns the fitness state

Return type float

controller.utility.**update_radio_name**(*radio*, *name*)

Change the name of the radio

Parameters

- **radio** (*string*) – radio serial number
- **name** (*string*) – name to associate with the radio

Returns True if the operation is successful, otherwise False

Return type boolean

controller.utility.**update_radio_transmit_power**(*radio*, *tr*)
Change the transmit-power of the radio

Parameters

- **radio** (*str*) – radio serial number
- **tr** (*int*) – new transmit power

Returns True or False

Return type bool

controller.utility.**update_send_time**(*idv*, *st*)

Parameters

- **idv** (*str*) – plant serial number
- **st** (*int*) – send time interval, minutes

controller.utility.**update_sleep_time**(*radio*, *time*)
Change the sleep-time of the radio

Parameters

- **radio** (*str*) – radio serial number
- **time** (*int*) – new sleep time (minutes)

Returns True or False

Return type bool

controller.utility.**update_temp**(*idv*, *temperature*)

Parameters

- **idv** (*str*) – plant serial number
- **temperature** (*int*) – temperature measure

controller.utility.**update_temp_max**(*idv*, *temp_m*)

controller.utility.**update_temp_min**(*idv*, *temp_m*)

controller.utility.**update_vase_transmit_power**(*idv*, *tr*)
Change the radio transmit power of the plant

Parameters

- **idv** (*str*) – serial number
- **tr** (*int*) – value of the new transmit power

Returns True or False

Return type bool

controller.utility.**update_water_container_size**(*idv*, *wcs*)

controller.utility.**update_water_container_state**(*idv*, *state*)

controller.utility.**update_watering_light**(*idv*, *wl*)

controller.utility.**url_from_plant**(*idv*)

Gets the url of the associated raspberry-radio.

Parameters **idv** (*str*) – plant serial number

Returns url or -1

Return type str

controller.utility.**url_from_radio**(*radio*)

Gets the radio url.

Parameters **radio** (*string*) – serial number

Returns the url of the radio or None

Return type string

CHAPTER

FOUR

CONTROLLER, REQUEST

controller.request.**request**()

Recive a request or response from a Radio-Raspberry

Data received:

- data['request']
- data['serial']

Optional Data received:

- data['signal']
- data['param']
- data['url']

Types of request:

- Operation.RADIO_JOIN:

  call the func. *add_radio(id, url)*, the id and url are data (json) received from the radio-raspberry, and if the radio has associated vases, send an http request to send information about them, so that the radio can add them to the list.

- Operation.CONNECTION

  call the func. *add_conn_req* with the received data.

- Operation.REFUSED

  call the func. *delete_conn_req*

- Operation.JOINED

  call the func. *add_plant*

- Operation.DELETED

  call the func. *delete_plant*

- Operation.HUMIDITY

  call the func. *update_hum(serial, param)* and *update_plant_state_fitness(serial)*

- Operation.LIGHT

  call the func. *update_light(serial, param)* and *update_plant_state_fitness(serial)*

- Operation.TEMPERATURE

  call the func. *update_temp(serial, param)* and *update_plant_state_fitness(serial)*

- Operation.SET_WATERING_LIGHT

  call the func. *update_watering_light(serial, param)*

- Operation.WATER_CONTAINER_STATE

  call the func. *update_water_container_state(serial, param)*

- Operation.SET_WATER_CONTAINER_SIZE

  call the func. *update_water_container_size(serial, param)*

- Operation.SET_HUMIDITY_MIN

  call the func. *update_hum_min(serial, param)* and *update_plant_state_fitness(serial)*

- Operation.SET_HUMIDITY_MAX

  call the func. *update_hum_max(serial, param)* and *update_plant_state_fitness(serial)*

- Operation.SET_LIGHT_MIN

  call the func. *update_light_min(serial, param)* and *update_plant_state_fitness(serial)*

- Operation.SET_LIGHT_MAX

  call the func. *update_light_max(serial, param)* and *update_plant_state_fitness(serial)*

- Operation.SET_TEMPERATURE_MIN

  call the func. *update_temp_min(serial, param)* and *update_plant_state_fitness(serial)*

- Operation.SET_TEMPERATURE_MAX

  call the func. *update_temp_max(serial, param)* and *update_plant_state_fitness(serial)*

- Operation.VASE_TRANSMIT_POWER

  call the func. *update_vase_transmit_power(serial, param)*

- Operation.RADIO_TRANSMIT_POWER

  call the func. *update_radio_transmit_power(serial, param)*

- Operation.SET_VASE_SEND_TIME

  call the func. *update_send_time(serial, param)*

- Operation.SET_RADIO_PAUSE_TIME

  call the func. *update_sleep_time(serial, param)*

CHAPTER

FIVE

VIEW, LOGIN PAGE

view.login.**is_admin**()
If the current user is authenticated and is admin return True, otherwise False

Returns True or False

Return type bool

view.login.**load_user**(*user_id*)
Gets the user with id equal to user_id

Parameters **user_id** (*int*) – id

Returns user

Return type *User*

view.login.**login**()
If current user is already authenticated return redirect to the homepage, otherwise return the loginpage.

Returns render_template or redirect

Return type Response

view.login.**logout**(**args*, ***kwargs*)
Call the function *logout_user()*

Returns *redirect '/login'*

CHAPTER

SIX

VIEW, DASHBOARD PAGE

view.dashboard.**dash_page**(**args*, ***kwargs*)

Dashboard page, show a map and charts. The map was created with *arbor.js*.

Returns template *dashboard.html*

Return type template

CHAPTER

SEVEN

VIEW, CONNECTIONS PAGE

view.connections.**add_plant_from_conn**(*idv*, *radio_id*)

Add a plant from a connection request: sends a HTTP request to the radio-raspberry with serial n. equal to radio_id, to join the plant into its list, and sends another HTTP request to the others radio to refuse the plant.

Parameters

- **idv** (*str*) – plant serial number
- **radio_id** (*str*) – radio serial number

view.connections.**conn_page**(**args*, ***kwargs*)

Show all connection requests, if the user is Admin

Returns template connections.html

Return type template

view.connections.**refuse_plant_from_conn**(*idv*, *radio_id*)

Refuse a plant sending HTTP request to the radio with serial number equal to radio_id. If the radio-raspberry is unreachable then delete the connection request of the vase from this radio.

Parameters

- **idv** (*str*) – plant serial number
- **radio_id** (*str*) – radio serial number

CHAPTER

EIGHT

VIEW, PLANTS PAGE

view.plants.**plants**(**args*, ***kwargs*)

Shows all the plants in the *giovase.db*

Returns template *plants.html*

Return type template

CHAPTER

NINE

VIEW, PLANT PAGE

view.plant.**container_state**(**args*, ***kwargs*)
Send an http request to the radio-raspberry associated with the plant,

to request the *WATER_CONTAINER_STATE* Operation

Parameters **idv** (*str*) – plant serial number

Returns redirect to the plantpage

Return type Response

view.plant.**humidity**(**args*, ***kwargs*)
Send an http request to the radio-raspberry associated with the plant,

to request the *HUMIDITY* Operation

Parameters **idv** (*str*) – plant serial number

Returns redirect to the plantpage

Return type Response

view.plant.**light**(**args*, ***kwargs*)
Send an http request to the radio-raspberry associated with the plant,

to request the *LIGHT* Operation

Parameters **idv** (*str*) – plant serial number

Returns redirect to the plantpage

Return type Response

view.plant.**plant_page**(**args*, ***kwargs*)
Show the plant page with id equal to idv. The page will contain the vital values of the plant and buttons to modify/delete the plant or request operations on it.

Parameters **idv** (*str*) – plant serial number

Returns template *plant.html*

Return type template

view.plant.**temperature**(**args*, ***kwargs*)
Send an http request to the radio-raspberry associated with the plant,

to request the *TEMPERATURE* Operation

Parameters **idv** (*str*) – plant serial number

Returns redirect to the plantpage

Return type Response

view.plant.**vase_state_req**(**args*, ***kwargs*)

Send three http request to the radio-raspberry associated with the plant,

to request the *TEMPERATURE*, *LIGHT*, *HUMIDITY* Operations

Parameters **idv** (*str*) – plant serial number

Returns redirect to the plantpage

Return type Response

view.plant.**water**(**args*, ***kwargs*)

Send an http request to the radio-raspberry associated with the plant,

to request the *WATER* Operation

Parameters **idv** (*str*) – plant serial number

Returns redirect to the plantpage

Return type Response

CHAPTER

TEN

VIEW, PLANTSETTINGS PAGE

view.plant_settings.**settings**(**args*, ***kwargs*)

If the current user is admin, show the plant settings.

Parameters **idv** (*str*) – plant serial number

Returns template *plant_settings.html* or *error.html*

Return type template

CHAPTER

ELEVEN

VIEW, RADIOS PAGE

view.radios.**radios**(**args*, ***kwargs*)

If the current user is admin, show all radios in the *giovase.db*.

Returns template *radios.html*

Return type template

CHAPTER

TWELVE

VIEW, RADIOSETTINGS PAGE

view.radio_settings.**radio_settings**(**args*, ***kwargs*)

If current user is admin, show the radio idr settings

Parameters **idr** (*str*) – radio serial number

Returns template *radio_settings.html*

Return type template

CHAPTER

THIRTEEN

VIEW, USERS PAGE

view.users.**users_list_page**(**args*, ***kwargs*)

If current user is admin, show all the users.

Returns template *users.html*

Return type template

CHAPTER

FOURTEEN

VIEW, USERSETTINGS PAGE

view.user_settings.**user_settings**(**args*, ***kwargs*)

If current user is admin, show the user's settings.

Parameters **user** (*str*) – username

Returns template *user_settings.html* or *error.html*

Return type template

CHAPTER

FIFTEEN

VIEW, REGISTER USER PAGE

view.add_user.**add_user_page**(**args*, ***kwargs*)

Register user page

Returns template *add_user.html*

Return type template

PYTHON MODULE INDEX

INDEX

S

T

U

V

W